

Databases Assignment 1

QUICKBITES – FOOD DELIVERY SYSTEM

Database Design and Conceptual Modelling Report

Assignment 1 – Module A & Module B

Course: Database Systems

Instructor: Dr. Yogesh K. Meena

1. SYSTEM OVERVIEW

QuickBites is a database-driven food delivery management system designed to connect customers, restaurants, and delivery partners on a single digital platform. The system enables customers to browse restaurants, place orders, apply offers, make payments, track deliveries in real time, and provide reviews.

The primary objective of this system is to implement a robust, normalized relational database that ensures data consistency, integrity, and scalability while reflecting real-world food delivery operations.

The system integrates:

- User management
- Restaurant and menu management
- Order processing
- Payment handling
- Delivery tracking
- Rating and review management

The database design strictly follows relational modelling principles and enforces primary keys, foreign keys, referential integrity, and logical constraints.

2. PROJECT OBJECTIVES

The objectives of QuickBites are:

1. To design a normalized relational database schema.
2. To enforce referential integrity using primary and foreign keys.
3. To implement role-based operations (Customer, Restaurant Owner, Delivery Partner).
4. To support real-world food ordering and delivery workflows.
5. To create UML and ER diagrams reflecting the database schema.
6. To validate logical constraints such as order status tracking and payment validation.
7. To populate meaningful real-life sample data (10–20 rows per table).

3. FUNCTIONAL DESCRIPTION

The QuickBites system is designed to simulate a real-world food delivery platform. The following major functionalities are implemented and supported by the database schema.

3.1 User Registration and Role Management

The system supports three types of users: Customer, Restaurant Owner, and Delivery Partner.

All users are initially stored in the **Member** table, which contains common details such as name, email, and contact number.

Based on their role, additional information is stored in separate tables (Customer, RestaurantOwner, DeliveryPartner).

This design follows the specialization principle and avoids data redundancy while allowing future expansion of roles.

3.2 Restaurant and Menu Management

Restaurant owners can manage restaurants and their menu items.

Each restaurant can have multiple menu items, and each menu item belongs to a specific category.

The system stores restaurant details, menu item information, pricing, and availability status.

This structure enables organized menu management and efficient categorization of food items.

3.3 Order Placement and Processing

Customers can place orders by selecting multiple menu items from a restaurant.

An order includes customer details, restaurant details, delivery address, applied offer (if any), order status, and total amount.

Since one order can contain multiple items, and one item can appear in multiple orders, the **OrderItem** table is used to manage this many-to-many relationship.

The system tracks the order status throughout its lifecycle.

3.4 Offer and Discount Management

The system allows customers to apply promotional offers during order placement.

Each offer contains details such as discount type, discount value, and minimum order amount.

Offers are applied only when eligibility conditions are satisfied, ensuring logical validation within the system.

3.5 Payment Processing

Each order has a corresponding payment record.

The system stores payment mode, payment status, amount, and refund information (if applicable).

This ensures proper tracking of financial transactions related to each order.

3.6 Delivery Management

After order confirmation, a delivery partner is assigned.

The system records delivery status and delivery time.

Each delivery is linked to both the order and the assigned delivery partner, ensuring clear tracking of responsibilities.

3.7 Live Location Tracking

The system supports live tracking of deliveries.

Each delivery can have multiple location updates recorded over time.

This feature allows customers to monitor the progress of their order in real time.

3.8 Review and Rating System

After successful delivery, customers can provide ratings and feedback for both the restaurant and the delivery partner.

The system stores review details and rating values to maintain service quality and performance evaluation.

4. ROLES AND ROLE-BASED OPERATIONS

4.1 Member (Base Entity)

All users are stored in the Member table.

Attributes:

- member_id (PK)
- name
- email

- contact_number
- age
- image_url

4.2 Customer

Operations:

- Register and login
- Add multiple addresses
- Browse restaurants
- Place orders
- Apply offers
- Make payments
- Track deliveries
- Give food and delivery reviews

4.3 Restaurant Owner

Operations:

- Manage restaurant details
- Add/edit menu items
- Categorize menu items
- Update availability
- View orders

4.4 Delivery Partner

Operations:

- Accept delivery assignments
- Update delivery status
- Share live location
- Receive delivery ratings

5. USE CASE DESCRIPTION

Main Use Case: Place an Order

1. Customer selects restaurant.
2. Customer selects menu items.
3. System creates an Order.
4. OrderItems are created (resolving M:N).
5. Offer (optional) is applied.
6. Payment is processed.
7. Delivery is assigned.
8. DeliveryLocation records live tracking.
9. Customer gives review after completion.

6. DATA FLOW EXPLANATION

1. Member → Customer / Owner / Partner (Role specialization)
2. Customer → Order
3. Order → OrderItem

4. Order → Payment
5. Order → Delivery
6. Delivery → DeliveryLocation
7. Order → Reviews

Data flows from user interaction to order processing, payment confirmation, delivery tracking, and review storage.

7. DATABASE TABLE DESCRIPTION

Below are the tables with primary and foreign keys:

1. Member

PK: member_id

Stores common user details.

2. Customer

PK: customer_id

FK: member_id → Member

3. RestaurantOwner

PK: owner_id

FK: member_id → Member

4. Delivery Partner

PK: partner_id

FK: member_id → Member

5. Restaurant

PK: restaurant_id
FK: owner_id → RestaurantOwner

6. Address

PK: address_id
FK: customer_id → Customer

7. Category

PK: category_id

8. MenuItem

PK: item_id
FK: restaurant_id → Restaurant
FK: category_id → Category

9. Orders

PK: order_id
FK: customer_id → Customer
FK: restaurant_id → Restaurant
FK: address_id → Address
FK: offer_id → Offer

10. OrderItem

PK: order_item_id
FK: order_id → Orders
FK: item_id → MenuItem

Resolves M:N between Orders and MenuItem.

11. Offer

PK: offer_id

12. Payment

PK: payment_id
FK: order_id → Orders

13. Delivery

PK: delivery_id
FK: order_id → Orders
FK: partner_id → DeliveryPartner

14. DeliveryLocation

PK: location_id
FK: delivery_id → Delivery

Uses composition relationship.

15. FoodReview

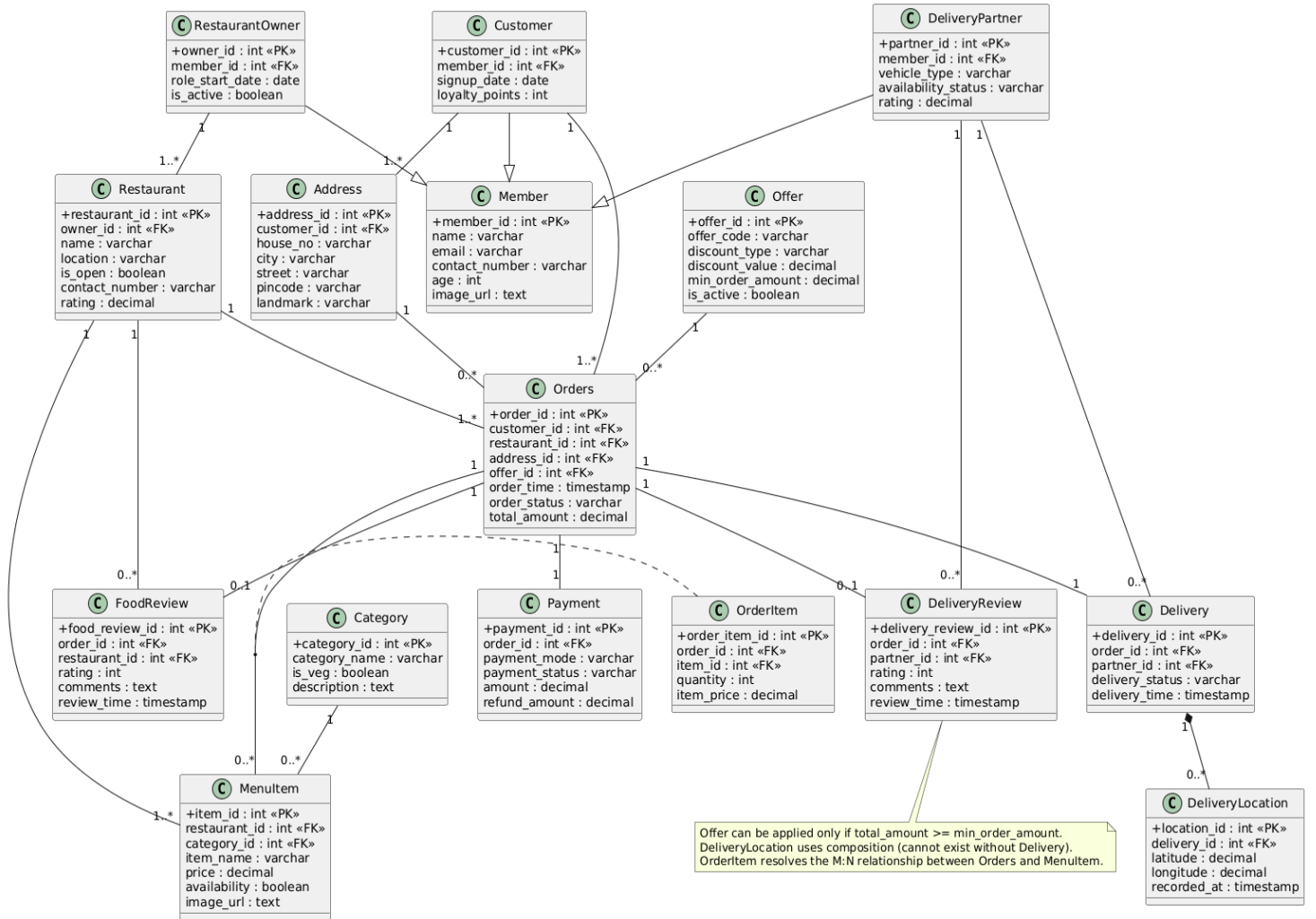
PK: food_review_id
FK: order_id → Orders
FK: restaurant_id → Restaurant

16. Delivery Review

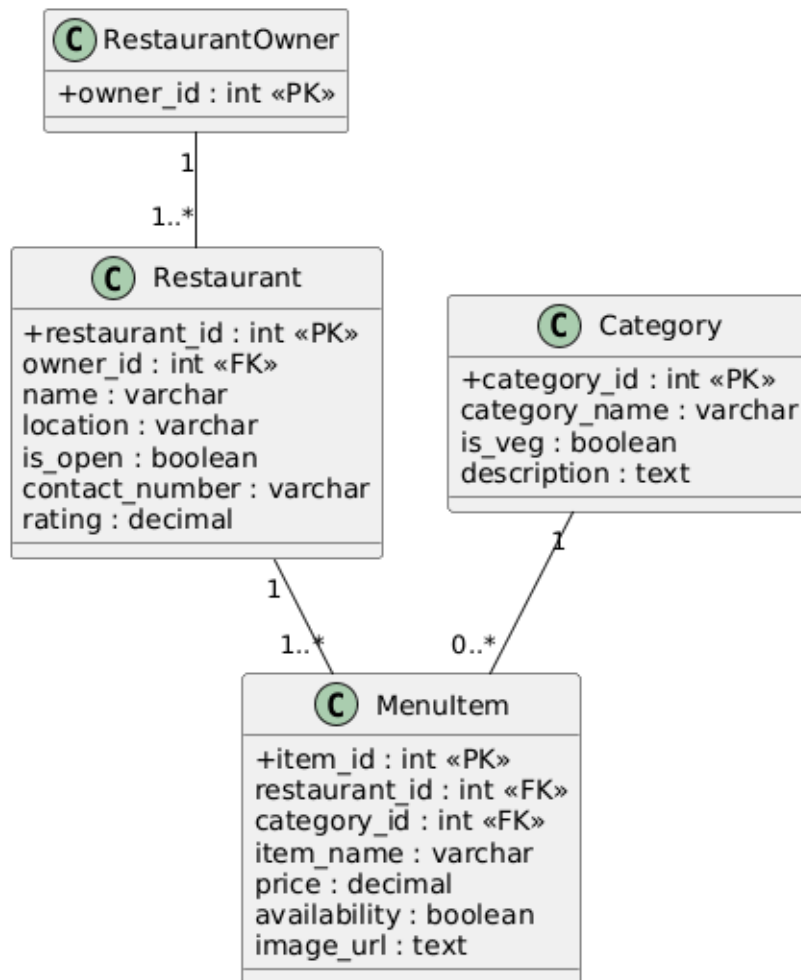
PK: delivery_review_id
FK: order_id → Orders
FK: partner_id → DeliveryPartner

8. UML DIAGRAM (Conceptual Representation)

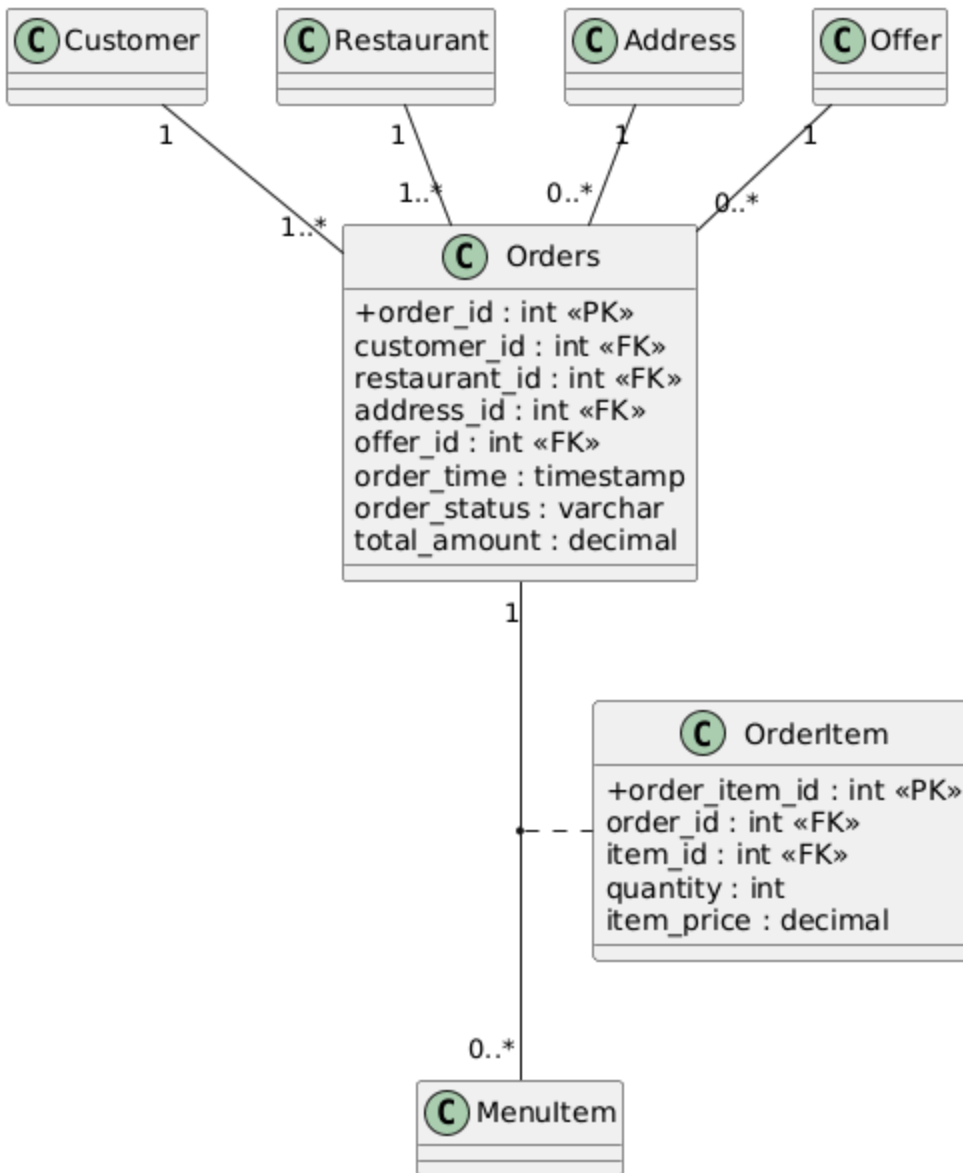
QuickBites - Complete UML Class Diagram (Updated Schema)



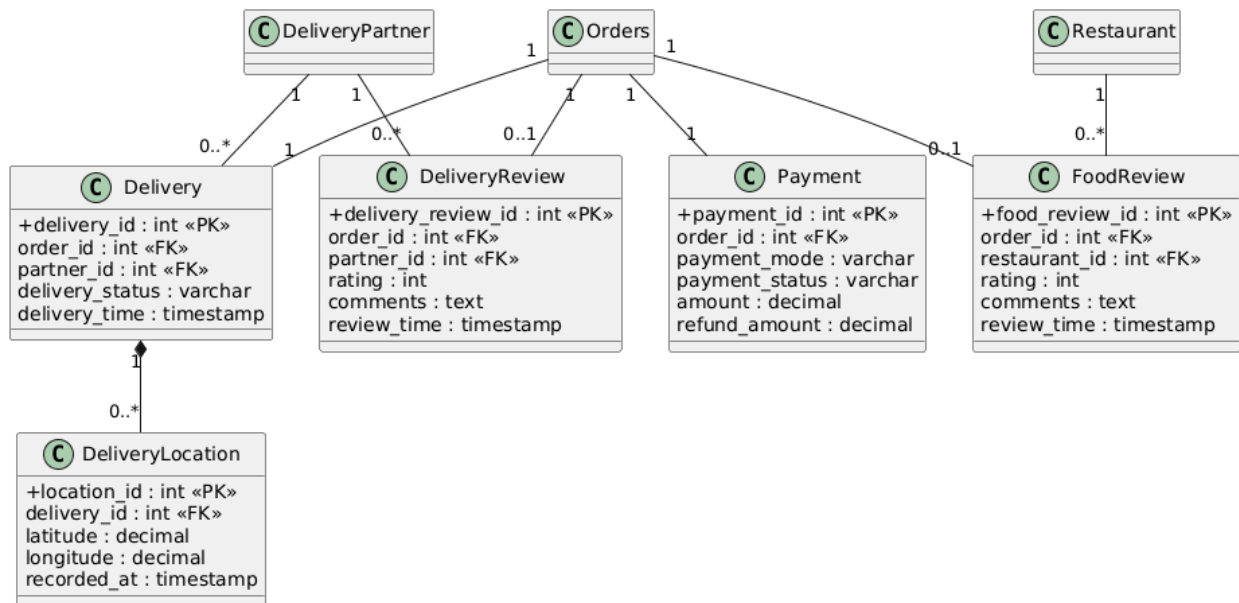
QuickBites - Restaurant & Menu Structure



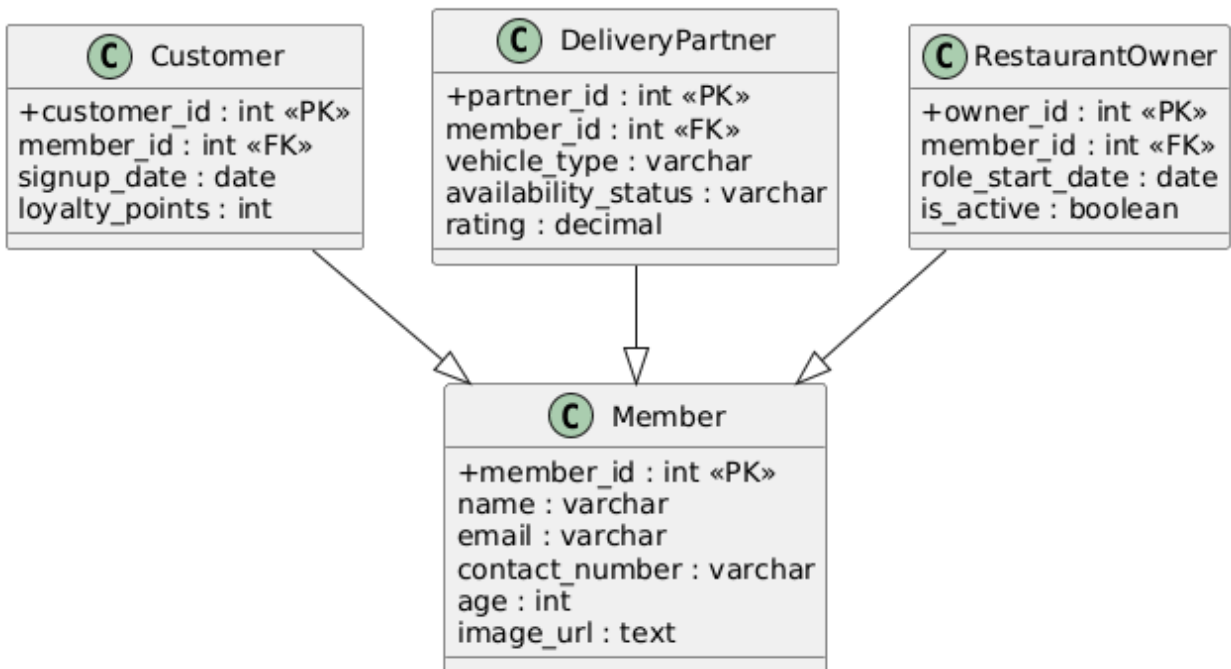
QuickBites - Order & Offer Structure



QuickBites - Payment, Delivery & Reviews



QuickBites - User Role Structure



Explanation:

- Member is a base class.

- Customer, RestaurantOwner, DeliveryPartner inherit from Member.
- Orders are associated with Customer and Restaurant.
- OrderItem resolves many-to-many.
- DeliveryLocation is composition of Delivery.

9. ER DIAGRAM (Conceptual Model)

Explanation:

- Entities represented as rectangles.
- Relationships represented as diamonds.
- Primary keys underlined.
- Foreign keys clearly marked.
- Cardinalities defined (1:M, M:N).

10. UML → ER TRANSITION EXPLANATION

The transition from UML to ER model was performed as follows:

1. **UML Classes → ER Entities**
Each class in UML (Member, Customer, Orders, etc.) became an ER entity.
2. **UML Associations → ER Relationships**
Associations between classes were converted into ER relationships.
3. **Inheritance Mapping**
The specialization of Member into Customer, RestaurantOwner, and DeliveryPartner was implemented using separate tables with foreign keys

referencing Member.

4. **Many-to-Many Resolution**

The M:N relationship between Orders and MenuItem was resolved using the associative entity OrderItem.

5. **Composition Mapping**

DeliveryLocation is dependent on Delivery and cannot exist independently. Therefore, in ER, DeliveryLocation includes a foreign key referencing Delivery with referential integrity enforced.

6. **Multiplicity Conversion**

UML multiplicity (1..*, 0..1) was converted into foreign key constraints in relational schema.

11. RELATIONSHIP JUSTIFICATIONS

Customer – Address (1:M)

One customer can have multiple addresses.

RestaurantOwner – Restaurant (1:M)

One owner can own multiple restaurants.

Restaurant – MenuItem (1:M)

One restaurant can have multiple menu items.

Category – MenuItem (1:M)

One category can contain many menu items.

Customer – Orders (1:M)

One customer can place many orders.

Restaurant – Orders (1:M)

One restaurant can receive many orders.

Address – Orders (1:M)

One address can be used in many orders (delivery address).

Offer – Orders (1:M)

One offer can be used in many orders.

Orders – Payment (1:1)

Each order has exactly one payment record.

Orders – Delivery (1:1)

Each order is assigned exactly one delivery.

DeliveryPartner – Delivery (1:M)

One delivery partner can handle many deliveries.

Delivery – DeliveryLocation (1:M)

One delivery has many tracking location updates.

12. ADDITIONAL CONSTRAINTS

Logical constraints implemented:

- rating BETWEEN 1 AND 5
- quantity > 0
- price > 0
- discount_value > 0
- total_amount ≥ 0
- amount ≥ 0
- min_order_amount ≥ 0
- refund_amount ≥ 0
- loyalty_points ≥ 0
- delivery_status in (ASSIGNED, PICKED_UP, OUT_FOR_DELIVERY, DELIVERED, FAILED)
- order_status in (PLACED, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCELLED)
- payment_status in (PENDING, SUCCESS, FAILED, REFUNDED)
- vehicle_type in (BIKE, SCOOTER, CAR, BICYCLE)

All tables contain at least three NOT NULL attributes.

Referential integrity is enforced using foreign keys.

13. MODULE A IMPLEMENTATION

The SQL dump file includes:

- CREATE TABLE statements
- Primary and Foreign key constraints
- CHECK constraints
- INSERT statements (10–20 rows per table)
- Referential integrity enforcement

The database schema satisfies normalization rules up to 3NF.

14. CONCLUSION

The QuickBites Food Delivery System successfully integrates conceptual modelling with relational database implementation. The system ensures data integrity, scalability, and logical consistency through normalized design and strict constraint enforcement.

The UML and ER diagrams accurately represent the implemented schema, and all relationships are justified based on real-world food delivery operations.

This project demonstrates strong understanding of database design principles, normalization, entity relationships, and system modelling.

15.Team Members and Contributions

Name	Roll No	Contribution
Pravallika Matha	24110198	Database design (Module A design)
Poladasu Tanmai Manjula	24110261	SQL Implementation (Module A – Implementation)

Modalavalasa Anusha	24110206	UML Diagrams (Module B – UML)
Reddyboina Triveni	24110295	ER Diagram (Module B – ER)
Gullapalli Kavya Durga Sri	24110125	LEAD (Integration & Documentation)